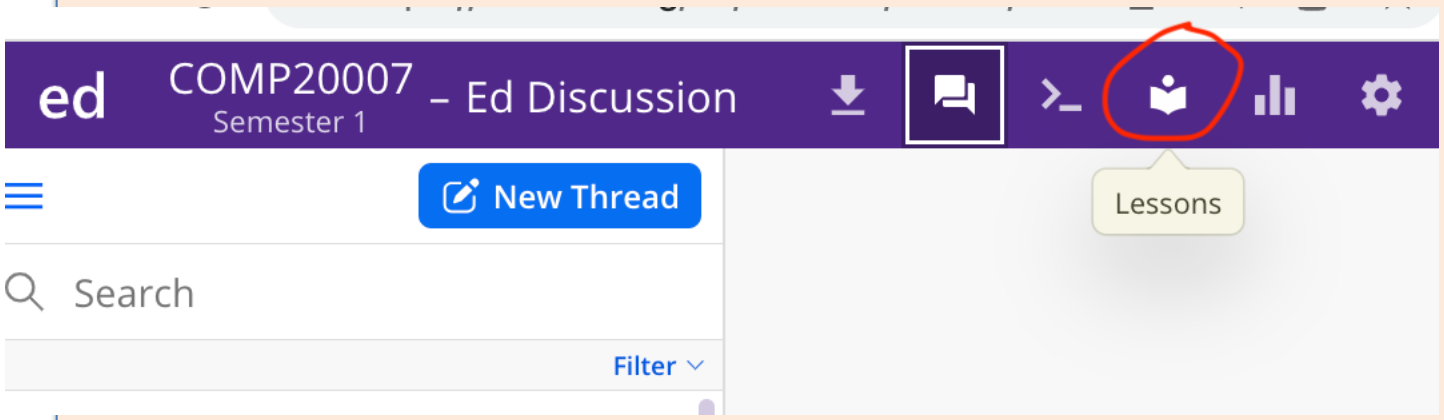
COMP20007 Workshop Week 1

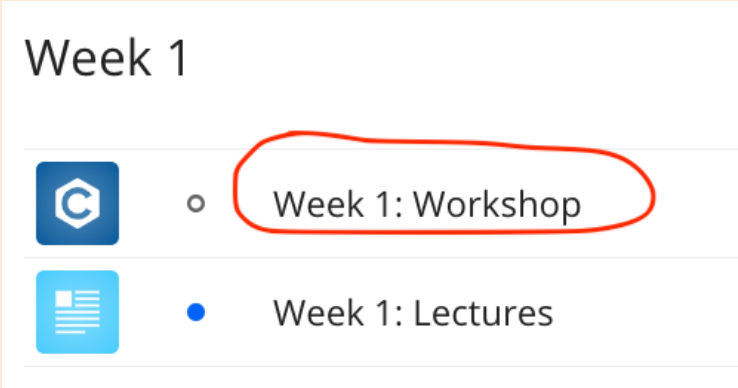
about Us and … C

While waiting:

- Talk to classmates, make friends
- Open LMS and click on "Ed Discussion" to open ED, on ED:

  ➢ click on Lessons → Week 1: Workshop

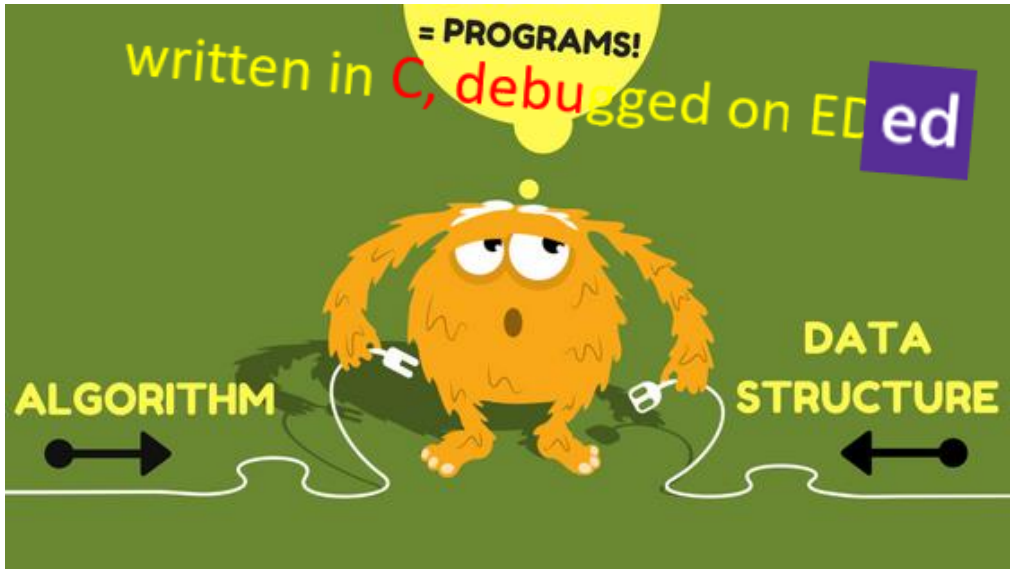




  ➢ Inspect the first 2 slides: "Tutorial" and "Tutorial: Slides"

Our Workshops

backup: the teaching team, ED, Internet, Google, GenAI, ...

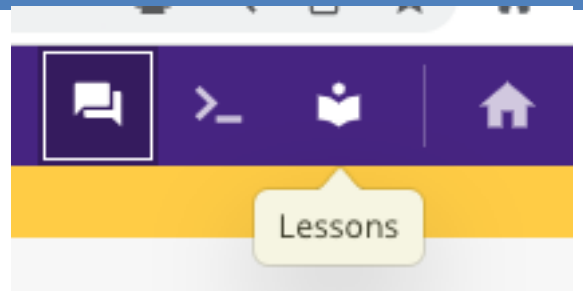


Bosses' Friend: Anh

avo@unimelb.edu.au

Our Platform: ED forum and lessons for week x (here, x == 1)

COMP20007 – Ed Discussion
Semester 1



Week 1

- Week 1: Workshop
- Week 1: Lectures

Materials for the Workshop of week x.

Use Tutorial: Slides as pre-workshop slides

Attempt all exercises (within the allotted time limit!)

Lecture slides and some resources.

Guided and Unguided lab

Work on each week's exercises

For this week, tutors will work through the first four exercises with you on Ed.

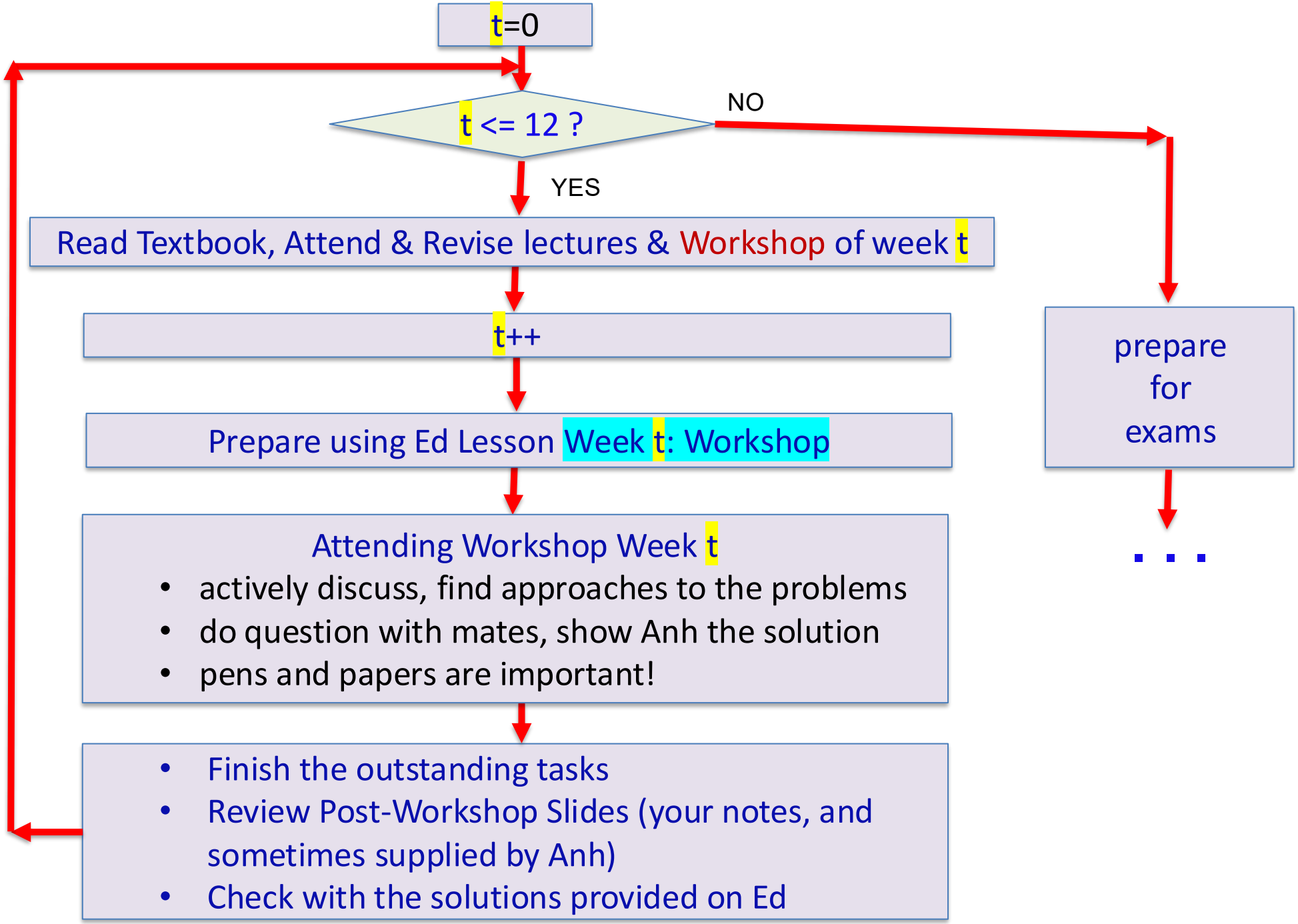
Generally, you will have the second part of the workshop to work unguided through the computer and tutorial question components of the workshop

In the 2nd hour, in general you work with your classmates on tutorial questions or programming exercises, show Anh your solutions and get feedbacks.

From next week: You should always have papers and pens ready!

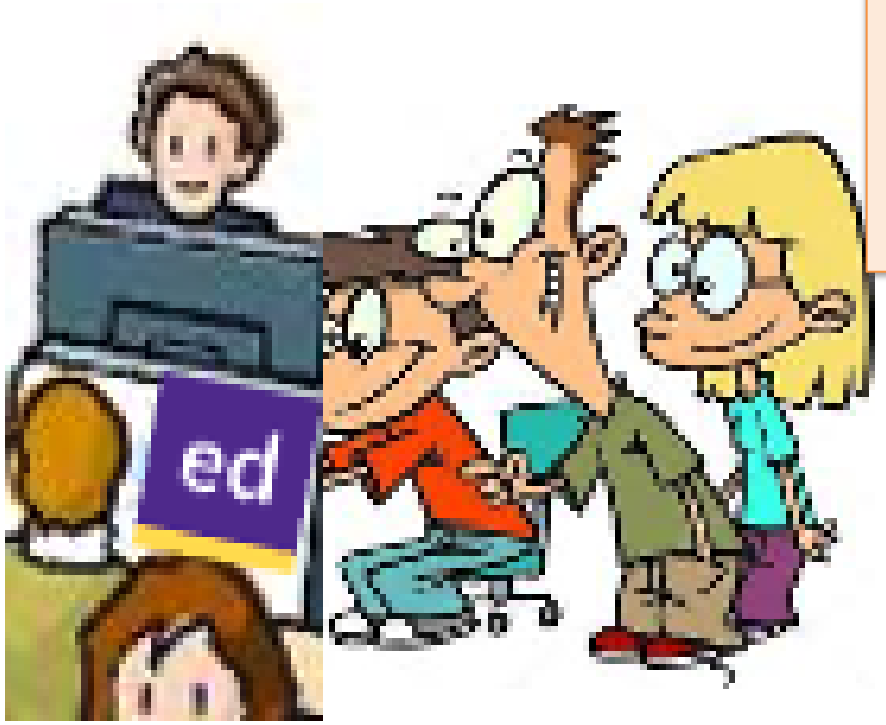
Problem:
Ace DoA

ALGORITHM



Now: time for (social) networking

- Engage in conversation
- Get to know your peers
- Share a joke or a fun fact about yourself
- Form your study groups!



What was the very first program you ever wrote?

If you had infinite time and O(1) effort, what app or software would you build?

What is the most frustrating bug you've ever had to fix?

Plan

- About Us
- C short review: Memory Management
- Survival Tools:
  - Essential Line Commands
  - Compiling, Executing
  - Debugging with valgrind & gdb

A Bit on C: Memory, Variables, Pointers

Two main categories of memory management in C:

- **Automatic Memory:**
  - Used for local variables, including function parameters.
  - Memory is allocated and deallocated automatically.
  - Accessed directly by their variable names.

- **Dynamic Memory:**
  - Accessed via pointers to memory chunks.
  - Allocated and deallocated explicitly by the programmer.

Visualisation: Memory Lane

House:

- Type
- Size
- Address

Memory Container:

- Type
- Size
- Address

							...
8 blocks		1 block		8 blocks		4 blocks	...
1 Foo St		13 Foo St		21 Foo St		42 Foo St	...
double	...	char	...	long	...	int	...
8 Bytes		1 Byte		8 Bytes		4 Bytes	...
0x075f5f		0x603e7a		0xbcde77		0xbd370a	...

Computer memory can be modelled as a single, vast array of bytes, where a memory address (such as **0x603e7a**) serves as the unique index for each individual byte.

Local Variables (Automatically-allocated in Stack Frames)

For the function **main()**, all its local variables (**a**, **b** and **sum**):

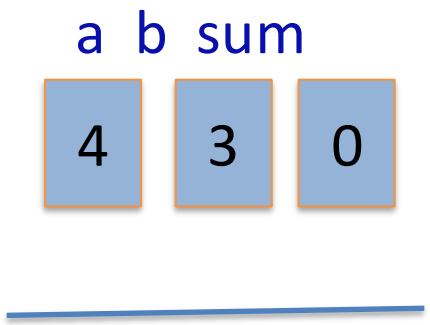
- are automatically allocated memory on the stack frame (SF) when the program starts,
- are automatically deallocated when the function exits.

main.c

```
f1  int foo(int m, int n) {
f2    int tmp;
f3    if (m > n) {
f4      tmp= m;
f5      m= n;
f6      n= tmp;
f7    }
      return m+n;
    }

m1  int main() {
m2    int a= 4, b= 3, sum=0;
m3    sum= foo(a, b);
m4    printf("%d %d %d\n",
            a, b, sum);
```

SF at line m2



Local Variables

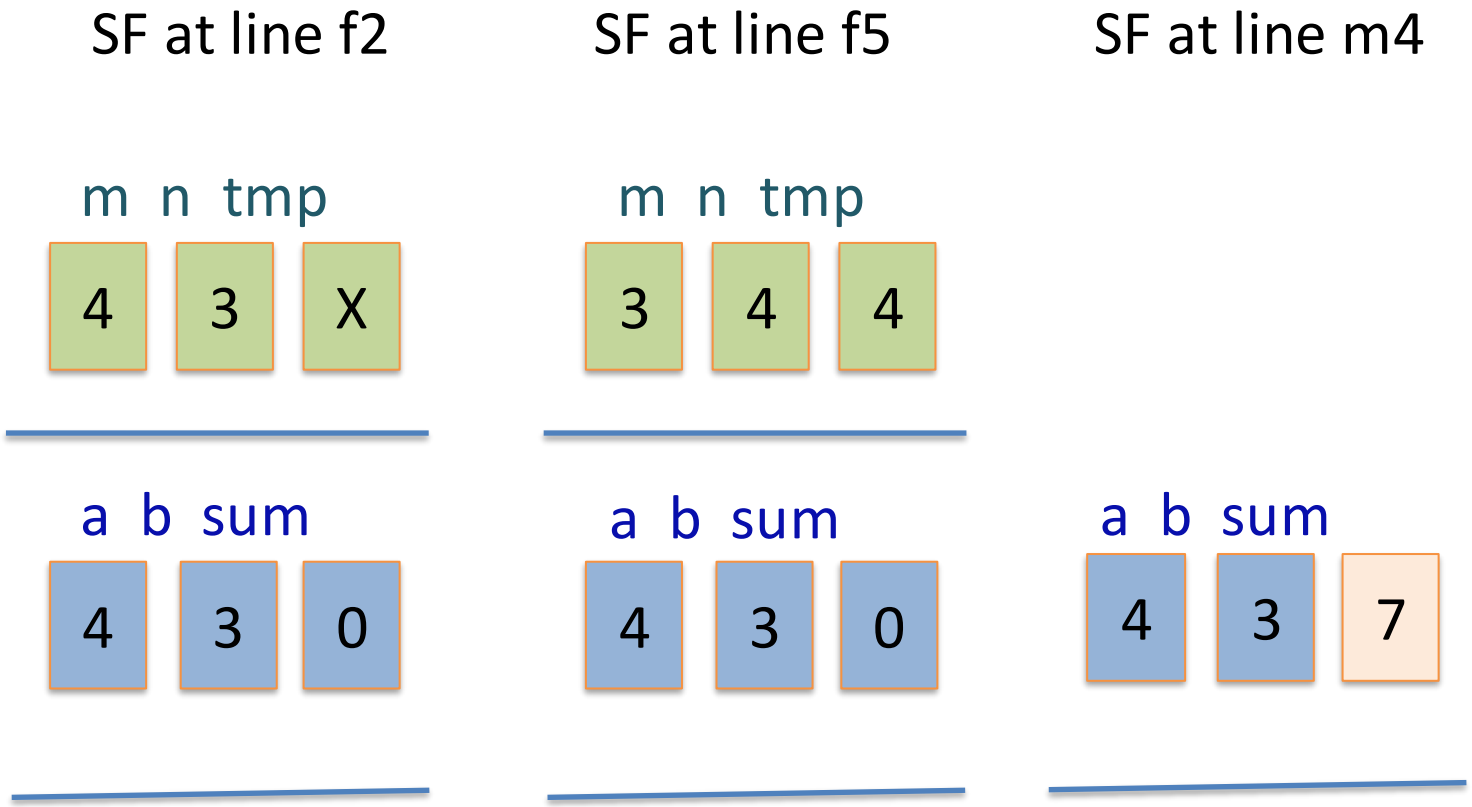
For a function (**foo**), all its local variables (**m** and **n**):

- are automatically allocated memory on the stack frame (SF) when the function is called,
- are automatically deallocated when the function exits.

main.c

```
f1  int foo(int m, int n) {
f2    int tmp;
f3    if (m > n) {
f4      tmp= m;
f5      m= n;
f6      n= tmp;
f7    }
      return m+n;
    }

m1  int main() {
m2    int a= 4, b= 3, sum=0;
m3    sum= foo(a, b);
m4    printf("%d %d %d\n",
            a, b, sum);
```



Local Variables

For a function (**foo**), all its local variables (**m** and **n**):

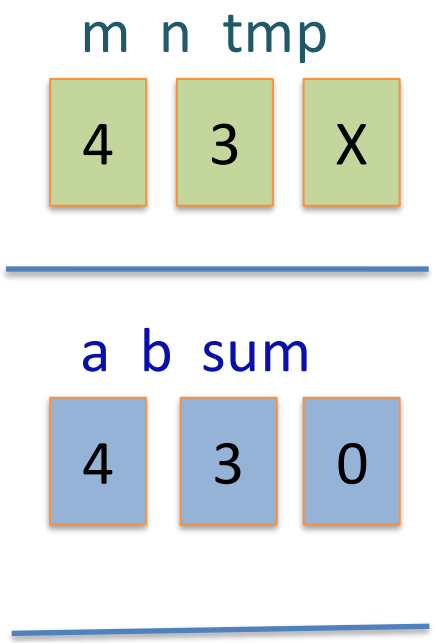
- are automatically allocated memory on the stack frame (SF) when the function is called,
- are automatically deallocated when the function exits.

main.c

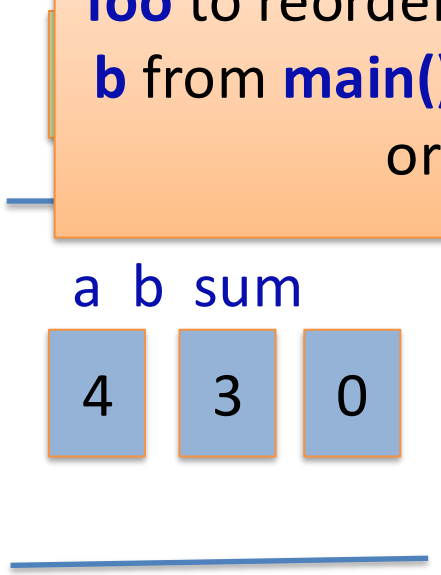
```
f1  int foo(int *m, int *n) {
f2    int tmp;
f3    if (*m > *n) {
f4      tmp= *m;
f5      *m= n;
f6      n= tmp;
f7    }
      return m+n;
    }
```

```
m1  int main() {
m2    int a= 4, b= 3, sum=0;
m3    sum= foo(&a, &b);
m4    printf("%d %d %d\n",
            a, b, sum);
```

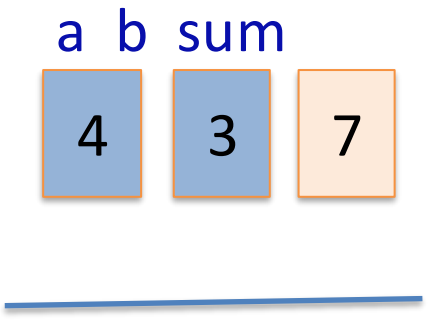
SF at line f2



SF at line f5



SF at line m4



How can we enable function **foo** to reorder variables **a** and **b** from **main()** into increasing order?

Solution: Using Pointers (note: when title starts with yellow, the page is a solution, *use it wisely!*)

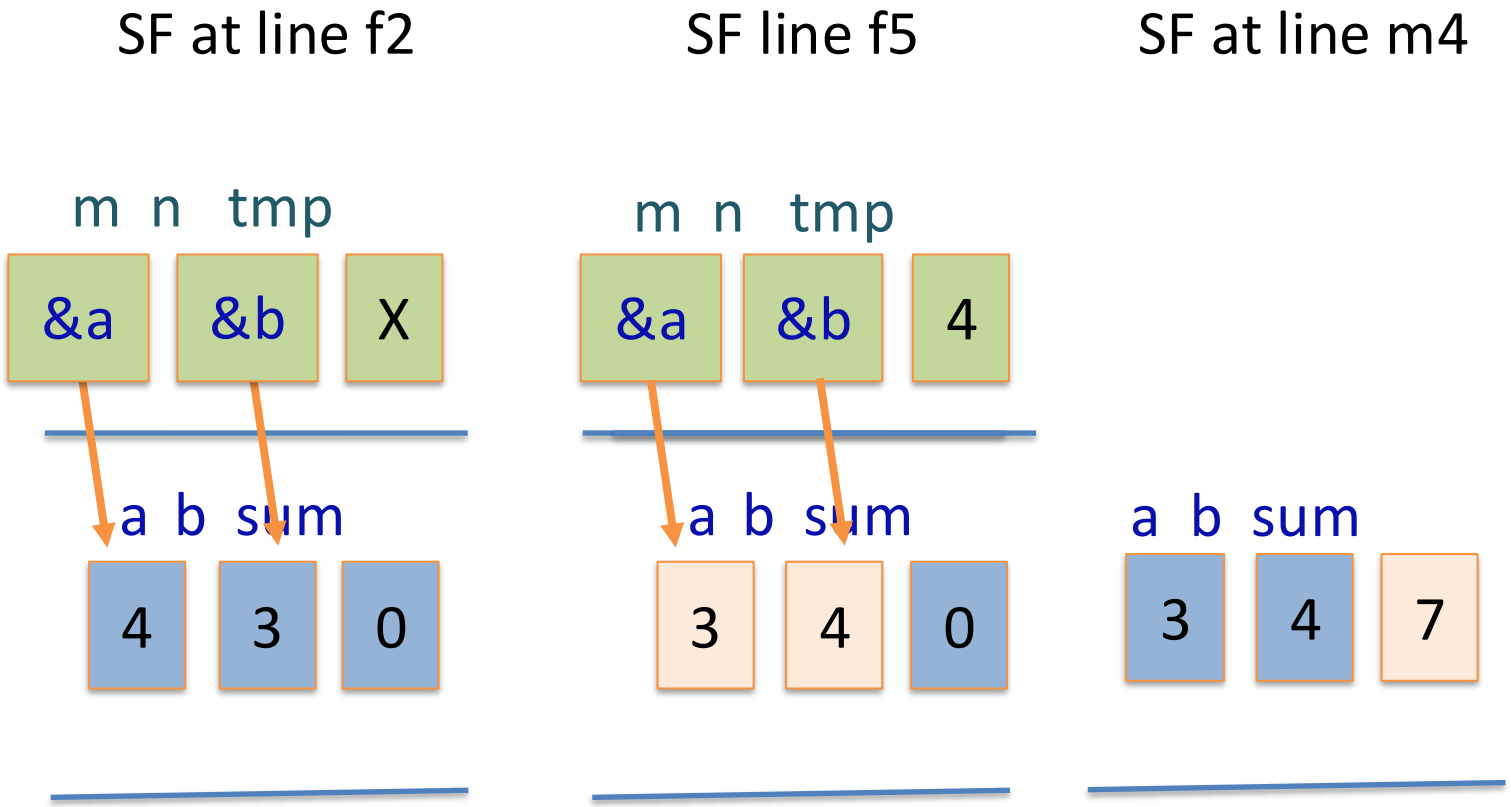
If function **foo** also wants to change **a** and **b** it needs to receive the *address* of a and b. The address **&a** and **&b** are called *pointers*. The type of **&a** and **&b** is **int ***

int foo(int m, int n)

```
f1  int foo(int *m, int *n) {
f2    int tmp;
f3    if (*m > *n) {
f4      tmp= *m;
f5      *m= *n;
f6      *n= tmp;
f7    }
      return *m + *n;
    }
```

```
m1  int main() {
m2    int a= 4, b= 3, sum=0;
m3    sum= foo(&a, &b);
m4    printf("%d %d %d\n",
            a, b, sum);
```

Here m and n are *pointer variables*, &a and &b are *pointer constants*.



Pointers: Examples of Declaration

```
/* pointer variable declarations */
<type> *ptr;  // run-of-the-mill variable
int *example;     // pointer to an int
char *example2;   // pointer to a char
int **example3;   // pointer to a [pointer to an int]

/* Array access via pointers */
int arr[9];
arr[5] == *(arr + 5);  // these are functionally equivalent
```

Remember? An array name is a pointer, specifying the address of the array's first element

Size of basic data types, including pointers

```c
/* Expected values in bytes
 * (within most systems, anyways)
 */
sizeof(char) == 1;
sizeof(float) == 4;
sizeof(int) == 4;
sizeof(double) == 8;
sizeof(long) == 8;
```

```c
/* All pointers have the same size
 * since addresses have equal sizes
 */
sizeof(int*) == 8;
sizeof(int**) == 8;
sizeof(char*) == 8;
```

Pointers: Examples with Memory Images

```
1  int main(void){
2    int *intPtr;
3    int number = 5;
4    int **intPtrPtr;
5    char *charPtr;
6    intPtrPtr = &intPtr;
7    *intPtrPtr = &number;
8    *intPtr = 10;
9    charPtr = (char *) intPtr;
10   charPtr[0] = 'a'; // a is 0x61 or 97
11   charPtr[1] = 'b'; // b is 0x62 or 98
12 }
```

Name	Address	Value	Memory (Bytes)							
intPtr	0x7fffffffffffe0b0									
number	0x7fffffffffffe0ac	5	05	00	00	00				
intPtrPtr	0x7fffffffffffe0b8									
charPtr	0x7fffffffffffe0c0									

Pointers: Examples with Memory Images

```
1  int main(void){
2    int *intPtr;
3    int number = 5;
4    int **intPtrPtr;
5    char *charPtr;
6    intPtrPtr = &intPtr;
7    *intPtrPtr = &number;
8    *intPtr = 10;
9    charPtr = (char *) intPtr;
10   charPtr[0] = 'a'; // a is 0x61 or 97
11   charPtr[1] = 'b'; // b is 0x62 or 98
12 }
```

Name	Address	Value	Memory (Bytes)							
intPtr	0x7fffffffffffe0b0									
number	0x7fffffffffffe0ac	5	05	00	00	00				
intPtrPtr	0x7fffffffffffe0b8	0x7fffffffffffe0b0	**b0**	**e0**	**ff**	**ff**	**ff**	**ff**	**ff**	**7f**
charPtr	0x7fffffffffffe0c0									

Pointers: Examples with Memory Images

```
1  int main(void){
2    int *intPtr;
3    int number = 5;
4    int **intPtrPtr;
5    char *charPtr;
6    intPtrPtr = &intPtr;
7    *intPtrPtr = &number;
8    *intPtr = 10;
9    charPtr = (char *) intPtr;
10   charPtr[0] = 'a'; // a is 0x61 or 97
11   charPtr[1] = 'b'; // b is 0x62 or 98
12 }
```

Name	Address	Value	Memory (Bytes)							
intPtr	0x7fffffffffffe0b0	0x7fffffffffffe0ac	**ac**	**e0**	**ff**	**ff**	**ff**	**ff**	**ff**	**7f**
number	0x7fffffffffffe0ac	5	05	00	00	00				
intPtrPtr	0x7fffffffffffe0b8	0x7fffffffffffe0b0	b0	e0	ff	ff	ff	ff	ff	7f
charPtr	0x7fffffffffffe0c0									

Pointers: Examples with Memory Images

```
1   int main(void){
2     int *intPtr;
3     int number = 5;
4     int **intPtrPtr;
5     char *charPtr;
6     intPtrPtr = &intPtr;
7     *intPtrPtr = &number;
8     *intPtr = 10;
9     charPtr = (char *) intPtr;
10    charPtr[0] = 'a'; // a is 0x61 or 97
11    charPtr[1] = 'b'; // b is 0x62 or 98
12  }
```

Name	Address	Value	Memory (Bytes)							
intPtr	0x7fffffffffffe0b0	0x7fffffffffffe0ac	ac	e0	ff	ff	ff	ff	ff	7f
number	0x7fffffffffffe0ac	10	**0a**	**00**	**00**	**00**				
intPtrPtr	0x7fffffffffffe0b8	0x7fffffffffffe0b0	b0	e0	ff	ff	ff	ff	ff	7f
charPtr	0x7fffffffffffe0c0									

Pointers: Examples with Memory Images

```
1   int main(void){
2     int *intPtr;
3     int number = 5;
4     int **intPtrPtr;
5     char *charPtr;
6     intPtrPtr = &intPtr;
7     *intPtrPtr = &number;
8     *intPtr = 10;
9     charPtr = (char *) intPtr;
10    charPtr[0] = 'a'; // a is 0x61 or 97
11    charPtr[1] = 'b'; // b is 0x62 or 98
12  }
```

Name	Address	Value	Memory (Bytes)							
intPtr	0x7fffffffffffe0b0	0x7fffffffffffe0ac	ac	e0	ff	ff	ff	ff	ff	7f
number	0x7fffffffffffe0ac	10	0a	00	00	00				
intPtrPtr	0x7fffffffffffe0b8	0x7fffffffffffe0b0	b0	e0	ff	ff	ff	ff	ff	7f
charPtr	0x7fffffffffffe0c0	0x7fffffffffffe0ac	**ac**	**e0**	**ff**	**ff**	**ff**	**ff**	**ff**	**7f**

Pointers: Examples with Memory Images

```
1  int main(void){
2    int *intPtr;
3    int number = 5;
4    int **intPtrPtr;
5    char *charPtr;
6    intPtrPtr = &intPtr;
7    *intPtrPtr = &number;
8    *intPtr = 10;
9    charPtr = (char *) intPtr;
10   charPtr[0] = 'a'; // a is 0x61 or 97
11   charPtr[1] = 'b'; // b is 0x62 or 98
12 }
```

Name	Address	Value	Memory (Bytes)							
intPtr	0x7fffffffffffe0b0	0x7fffffffffffe0ac	ac	e0	ff	ff	ff	ff	ff	7f
number	0x7fffffffffffe0ac	97	**61**	00	00	00				
intPtrPtr	0x7fffffffffffe0b8	0x7fffffffffffe0b0	b0	e0	ff	ff	ff	ff	ff	7f
charPtr	0x7fffffffffffe0c0	0x7fffffffffffe0ac	ac	e0	ff	ff	ff	ff	ff	7f

Pointers: Examples with Memory Images

```
1  int main(void){
2    int *intPtr;
3    int number = 5;
4    int **intPtrPtr;
5    char *charPtr;
6    intPtrPtr = &intPtr;
7    *intPtrPtr = &number;
8    *intPtr = 10;
9    charPtr = (char *) intPtr;
10   charPtr[0] = 'a'; // a is 0x61 or 97
11   charPtr[1] = 'b'; // b is 0x62 or 98
12 }
```

Name	Address	Value	Memory (Bytes)							
intPtr	0x7fffffffffffe0b0	0x7fffffffffffe0ac	ac	e0	ff	ff	ff	ff	ff	7f
number	0x7fffffffffffe0ac	25185	61	**62**	00	00				
intPtrPtr	0x7fffffffffffe0b8	0x7fffffffffffe0b0	b0	e0	ff	ff	ff	ff	ff	7f
charPtr	0x7fffffffffffe0c0	0x7fffffffffffe0ac	ac	e0	ff	ff	ff	ff	ff	7f

Dynamic Memory

Dynamic memory is allocated during the runtime on programmer's request

- the programmer request/allocate a memory chunk using malloc (or calloc)
- a call to malloc/calloc might fail, and the functions return NULL in this case
- when the memory chunk is no longer needed the programmer must de-allocate it using free.

✗	✗	✗	✓

int *p;
p= 10;
…
<end program>
	int *p;
	p= malloc(sizeof(int));
	p= 10;
	…
	<end program>
		int *p;
		p= malloc(sizeof(int));
		assert(p !=NULL);
		*p= 10;
		…
		<end program>
			int *p;
			p= malloc(sizeof(int));
			assert(p !=NULL);
			*p= 10;
			…
			free(p);
			<end program>

Memory before malloc	Memory after malloc, and just after *p= 10
p <undefined value>	p 10

Pointers and dynamic memory: related tools in a nutshell

Operation	How?
Pointer declaration for <type>	<type> *ptr;
Allocation, method 1	ptr= (<type>*) malloc(sizeof(<type>)); assert(ptr);
Allocation, method 2 (using automatic casting)	ptr= malloc(sizeof(*ptr)); assert(ptr);
Allocation, zero-initialised	ptr= calloc(1, sizeof(*ptr)); assert(ptr);
Deallocation	free(ptr);

String Processing in an example

Task: Write a function stringReverse that returns the reverse of a string. The segment

```
char *rev= stringReverse("ABC");
printf("%s\n", rev);
```

should print out:

```
CBA
```

```
char* stringReverse( char *s) {
  char reverse[100];

  ???

  return reverse;
}
```

Question: Is it correct? Any problem when stringReverse is called?

Peer Activity: String Duplication

Will this code snippet work as intended? Why or why not?

a. Yes, it will.

b. No, it will not.

```
...
char buf[MAX_LEN];  // string buffer
char **dups =       // array of strings
    (char**)malloc(INITIAL*sizeof(char*));
int num_strings = 0;
/* Read strings from stdin */
while (fgets(buf, MAX_LEN, stdin) != NULL) {
    /* NULL-terminate the string */
    if (buf[strlen(buf) - 1] == '\n')
        buf[strlen(buf) - 1] = '\0';
    /* Store it into the array */
    dups[num_strings] = buf;
    num_strings++;
}
...
```

Peer Activity: String Duplication

Will this code snippet work as intended? Why or why not?

a. Yes, it will.

b. No, it will not.

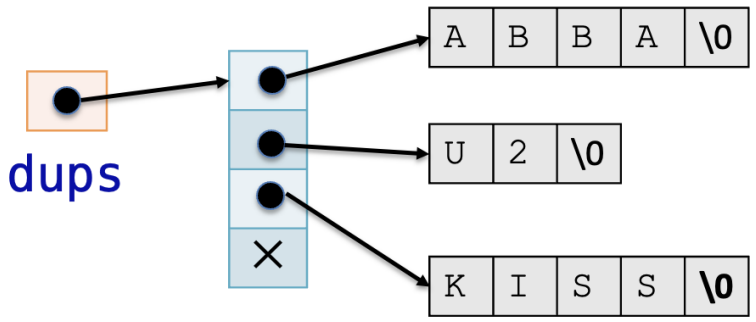
Supposing:

```
#define INITIAL 4
#define MAX_LEN 8
```

Inputs:

```
ABBA
U2
KISS
```

The Intention?




```
...
char buf[MAX_LEN];  // string buffer
char **dups =       // array of strings
    (char**)malloc(INITIAL*sizeof(char*));
int num_strings = 0;
/* Read strings from stdin */
while (fgets(buf, MAX_LEN, stdin) != NULL) {
    /* NULL-terminate the string */
    if (buf[strlen(buf) - 1] == '\n')
        buf[strlen(buf) - 1] = '\0';
    /* Store it into the array */
    dups[num_strings] = buf;
    num_strings++;
```

Peer Activity: String Duplication

Will this code snippet work as intended? Why or why not?

a. Yes, it will.

b. No, it will not.

```
...
char buf[MAX_LEN];  // string buffer
char **dups =       // array of strings
    (char**)malloc(INITIAL*sizeof(char*));
int num_strings = 0;
/* Read strings from stdin */
while (fgets(buf, MAX_LEN, stdin) != NULL) {
    /* NULL-terminate the string */
    if (buf[strlen(buf) - 1] == '\n')
        buf[strlen(buf) - 1] = '\0';
    /* Store it into the array */
    dups[num_strings] = buf;
    num_strings++;
}
...
```

Peer Activity: String Duplication

Will this code snippet work as intended? Why or why not?

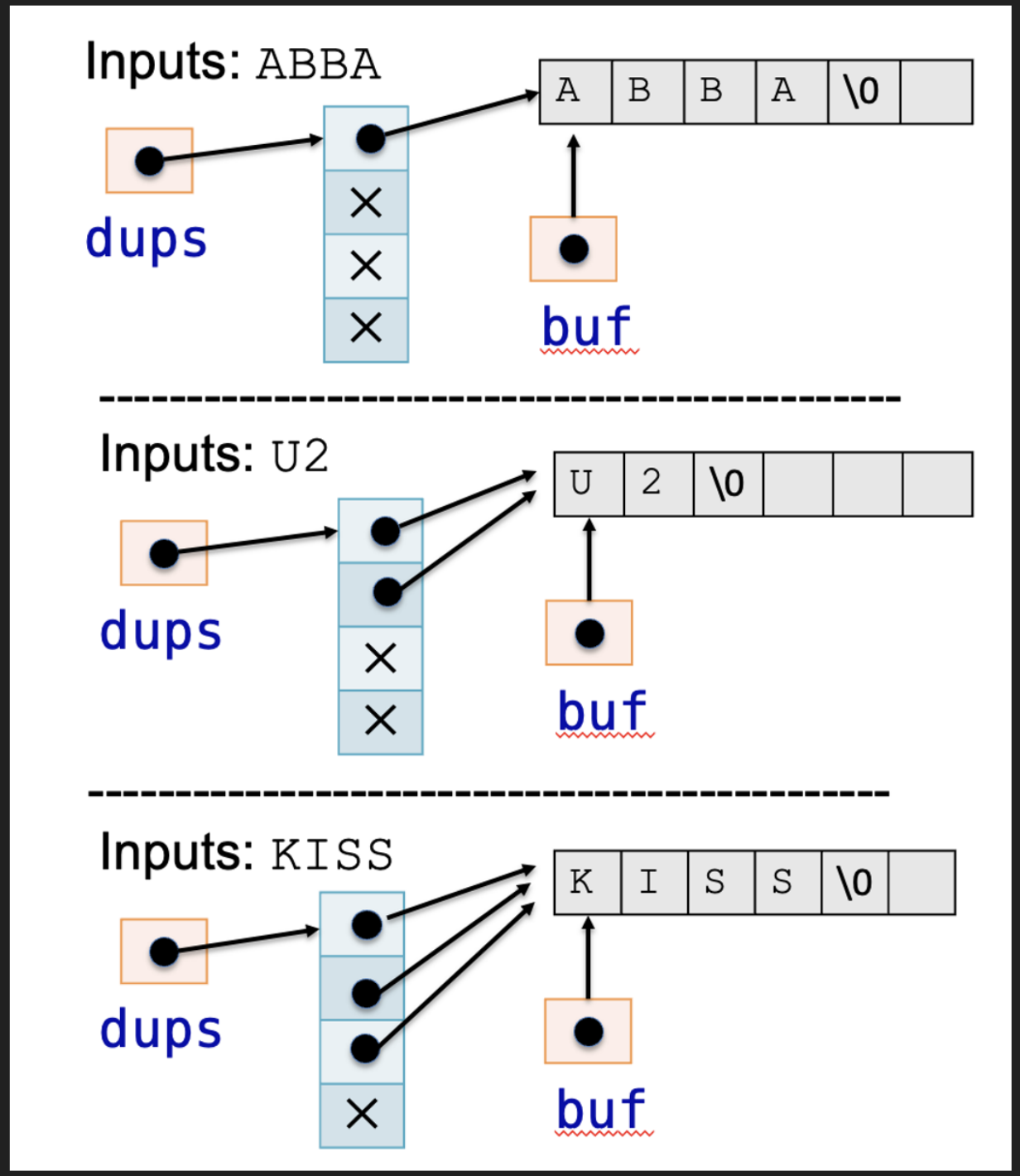
b. No, it will not.

Why?

- line 11 is a pointer assignment
- every element of dups[ ] will point to buf

How do we fix it?

```
...
 1 char buf[MAX_LEN];  // string buffer
 2 char **dups =       // array of strings
 3     (char**)malloc(INITIAL*sizeof(char*));
 4 int num_strings = 0;
 5 /* Read strings from stdin */
 6 while (fgets(buf, MAX_LEN, stdin) != NULL) {
 7     /* NULL-terminate the string */
 8     if (buf[strlen(buf) - 1] == '\n')
 9         buf[strlen(buf) - 1] = '\0';
10     /* Store it into the array */
11     dups[num_strings] = buf;
12     num_strings++;
13 }
...
```



What happened (MAX_LEN == 5)

```
...
char buf[MAX_LEN];  // string buffer
char **dups =       // array of strings
    (char**)malloc(INITIAL*sizeof(char*));
int num_strings = 0;
/* Read strings from stdin */
while (fgets(buf, MAX_LEN, stdin) != NULL) {
    /* NULL-terminate the string */
    if (buf[strlen(buf) - 1] == '\n')
        buf[strlen(buf) - 1] = '\0';
    /* Store it into the array */
    dups[num_strings] = buf;
    num_strings++;
}
...
```

How do we fix it?

Peer Activity: String Duplication

Will this code snippet work as intended? Why or why not?

b. No, it will not.

Why?

- line 11 is a pointer assignment
- every element of dups[ ] will point to buf

How do we fix it?

Answer:

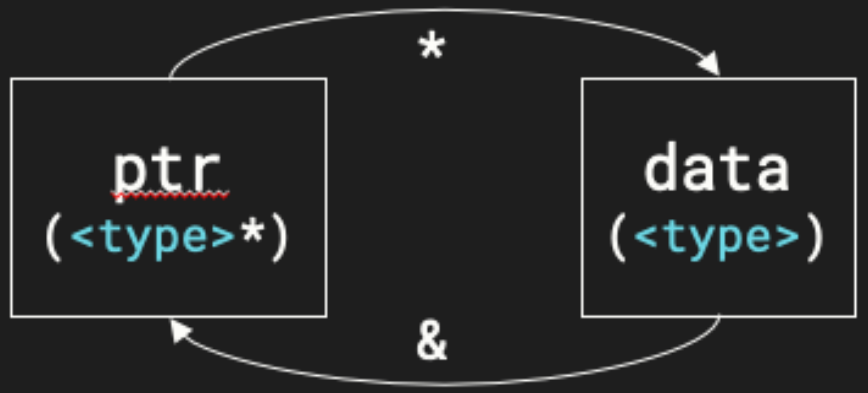
- `malloc()` + `strcpy()`
- `strdup()`

```
...
 1 char buf[MAX_LEN];  // string buffer
 2 char **dups =       // array of strings
 3     (char**)malloc(INITIAL*sizeof(char*));
 4 int num_strings = 0;
 5 /* Read strings from stdin */
 6 while (fgets(buf, MAX_LEN, stdin) != NULL) {
 7     /* NULL-terminate the string */
 8     if (buf[strlen(buf) - 1] == '\n')
 9         buf[strlen(buf) - 1] = '\0';
10     /* Store it into the array */
11     dups[num_strings] = strdup(buf);
12     num_strings++;
13 }
...
```

Pointers in a nutshell

Two basic **pointer operators**:

- **dereference operator** (*): "value pointed by"
- **reference operator** (&): "address of"



Operation	How?
Pointer declaration for <type>	<type> *ptr;
Allocation, method 1	ptr= (<type>*) malloc(sizeof(<type>)); assert(ptr);
Allocation, method 2 (using automatic casting)	ptr= malloc(sizeof(*ptr)); assert(ptr);
Allocation, zero-initialised	ptr= calloc(1, sizeof(*ptr)); assert(ptr);
Deallocation	free(ptr);

Discussion: storing a sequence of strings (supposing INITIAL large enough)

Will this code snippet work as intended? Why or why not?

a. Yes, it will.

b. No, it will not.

```
...
char buf[MAX_LEN];  // string buffer
char **dups =       // array of strings
    (char**)malloc(INITIAL*sizeof(char*));
int num_strings = 0;
/* Read strings from stdin */
while (fgets(buf, MAX_LEN, stdin) != NULL) {
    /* NULL-terminate the string */
    if (buf[strlen(buf) - 1] == '\n')
        buf[strlen(buf) - 1] = '\0';
    /* Store it into the array */
    dups[num_strings] = buf;
    num_strings++;
}
...
```

Program Development on ED

C Program Development Tools on ED

Program Development Tools

- Creating/Editing program.c (and other text files) using the excellent built-in editor.
- Compiling with:

```
gcc -Wall -g -o program program.c
```

- Using debugging tools such as valgrind:

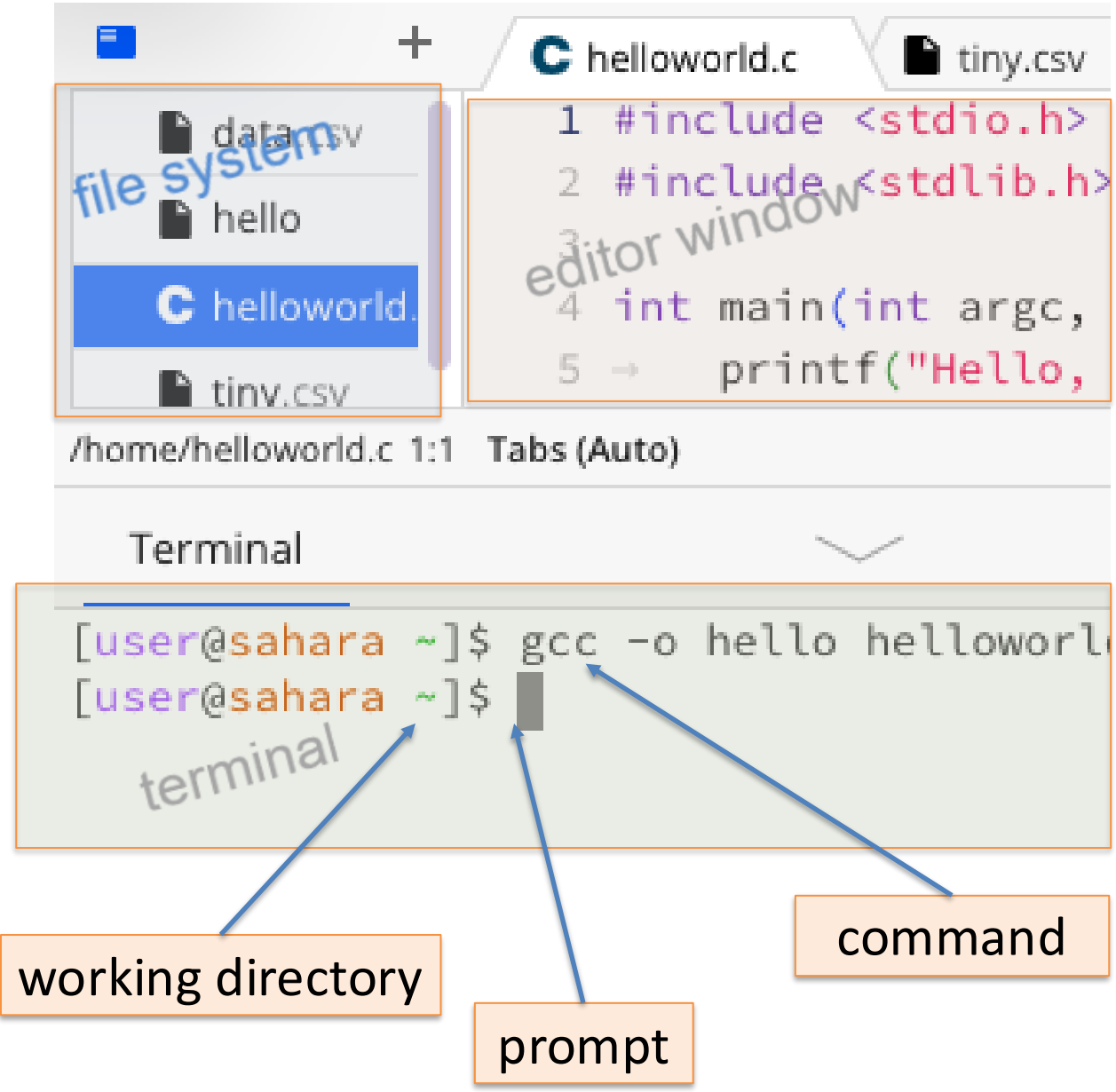
```
valgrind --leak-check=full --track-origins=yes ./program
```

That gives a report on current and/or potential issues of program.c, if any.

Debugging principles

- get no warnings/errors from gcc –Wall
- get a clean valgrind report
- when having problem, first debug with small and simple input
- having problems with larger inputs: focus on the first troublesome input part
- remember: all debug tools have limitations...

Using ED Workspace for a Program Project



A *workspace* is used for a programming project:

- It has a *file system*, starting with a *home directory* (~). Recursively, a directory can host files and its own sub-directories.
- The terminal allows us to interact with a Unix-style shell (namely bash).

Together with Tutor, do:

- Problem 1: essential shell commands
- Problem 2: valgrind
- Problem 3+4: using gdb

Lab

Appendices

Testing: Simplest Failing Test Case

Test code with the **simplest test case** that:

- you know should work, but
- does not work :')

For example,

Debugging principles

- get no warnings/errors from gcc –Wall
- get a clean valgrind report
- when having problem, first debug with small and simple input
- having problems with larger inputs: focus on the first troublesome input part
- remember: all debug tools have limitations...

Memory Profiling: Valgrind

Valgrind is a suite of dynamic analysis tools.

Memcheck is the default, memory-analysing tool and allows us to:

- see a program's cumulative memory utilisation
- decode arcane segmentation faults
- find **memory leaks:** manually-allocated memory that was never freed

Memory Profiling: Valgrind: Cheat Sheet

Synopsis:

```
valgrind [valgrind-options] your-program [your-program-options]
```

Essential options:

```
--leak-check=full     # shows details of each individual memory leaks
--track-origins=yes   # tracks the origins of uninitialised values
```

Note: This is an excerpt of Valgrind's man page.

Memory Profiling: Valgrind: Common Errors

Error message	Possible cause(s)
`Conditional jump or move depends on uninitialised value(s)`	Uninitialised values were used in the guard of `for`/`if`/`switch`/`while` statements
`Invalid free() / delete / delete[]`	Non-`*alloc()`'d memory was `free()`'d Some `*alloc()`'d memory was `free()`'d multiple times
`Invalid read/write of size X`	Accessing/modifying restricted memory
`Use of uninitialised value of size X`	Accessing/modifying uninitialised values

Common sizes: 1 (`char`), 2 (`short`), 4 (`float`, `int`), 8 (`double`, `long`, `<type>*`)

Memory Profiling: Valgrind: Demo

```
> nl demo.c
1 #include <stdio.h>
2 int main(void) {
3     int nums[5] =
          {0, 1, 2, 3, 4};
4     printf("5th num =
%d\n",
             nums[5]);
5     return 0;
6 }
```

```
# Compile the program with the debug flag
> gcc -Wall -g -o demo demo.c
# ...and run it through valgrind
> valgrind ./demo
==42== Memcheck, a memory error detector
...
==42== Command: ./demo
...
==42== Use of uninitialised value of size 8
==42==    at 0x48CF3CB: _itoa_word (in /usr/lib/libc-2.33.so)
==42==    by 0x48E8D9B: __vprintf_internal (in /usr/lib/libc-2.33.so)
==42==    by 0x48D52DE: printf (in /usr/lib/libc-2.33.so)
==42==    by 0x10919B: main (demo.c:4)
...
5th num = 32   # yet another whaaa?
...
==42== Use --track-origins=yes to see where uninitialised values come
from
...
```

Memory Profiling: Valgrind: Demo

```
> nl demo.c
1 #include <stdio.h>
2 int main(void) {
3     int nums[5] =
          {0, 1, 2, 3, 4};
4     printf("5th num =
%d\n",
            nums[5]);
5     return 0;
6 }
```

```
# Compile the program with the debug flag
> gcc -Wall -g -o demo demo.c
# ...and run it through valgrind
> valgrind ./demo
==42== Memcheck, a memory error detector
...
==42== Command: ./demo
...
==42== Use of uninitialised value of size 8
==42==    at 0x48CF3CB: _itoa_word (in /usr/lib/libc-2.33.so)
==42==    by 0x48E8D9B: __vprintf_internal (in /usr/lib/libc-2.33.so)
==42==    by 0x48D52DE: printf (in /usr/lib/libc-2.33.so)
==42==    by 0x10919B: main (demo.c:4)
...
5th num = 32   # yet another whaaa?
...
==42== Use --track-origins=yes to see where uninitialised values come
from
...
```

Memory Profiling: Valgrind: Demo

```
> nl demo.c
1 #include <stdio.h>
2 int main(void) {
3     int nums[5] =
          {0, 1, 2, 3, 4};
4     printf("5th num =
%d\n",
             nums[5]);
5     return 0;
6 }
```

```
# Compile the program with the debug flag
> gcc -Wall -g -o demo demo.c
# ...and run it through valgrind with the --track-origins=yes option
> valgrind --track-origins=yes ./demo
==42== Memcheck, a memory error detector
...
==42== Command: ./demo
...
==42== Use of uninitialised value of size 8
==42==    at 0x48CF3CB: _itoa_word (in /usr/lib/libc-2.33.so)
==42==    by 0x48E8D9B: __vprintf_internal (in /usr/lib/libc-2.33.so)
==42==    by 0x48D52DE: printf (in /usr/lib/libc-2.33.so)
==42==    by 0x10919B: main (demo.c:4)
==42==  Uninitialised value was created by a stack allocation
==42==    at 0x109149: main (demo.c:2)
...
5th num = 32   # srsly, whaaa?
...
```

Memory Profiling: Valgrind: Demo

```
> nl demo.c
1 #include <stdio.h>
2 int main(void) {
3     int nums[5] =
          {0, 1, 2, 3, 4};
4     printf("5th num =
%d\n",
            nums[5]);
5     return 0;
6 }
```

```
# Compile the program with the debug flag
> gcc -Wall -g -o demo demo.c
# ...and run it through valgrind with the --track-origins=yes option
> valgrind --track-origins=yes ./demo
==42== Memcheck, a memory error detector
...
==42== Command: ./demo
...
==42== Use of uninitialised value of size 8
==42==    at 0x48CF3CB: _itoa_word (in /usr/lib/libc-2.33.so)
==42==    by 0x48E8D9B: __vprintf_internal (in /usr/lib/libc-2.33.so)
==42==    by 0x48D52DE: printf (in /usr/lib/libc-2.33.so)
==42==    by 0x10919B: main (demo.c:4)
==42==  Uninitialised value was created by a stack allocation
==42==    at 0x109149: main (demo.c:2)
...
5th num = 32   # srsly, whaaa?
...
```

Daily Tool: Command-Line Interface

```
cat file_name    # prints file_name's contents to stdout
cd dir_path      # changes the current directory to dir_path
clear            # wipes the terminal clean
ls dir_path      # lists the contents of dir_path
mkdir dir_name   # creates a new directory dir_name
rm file_name     # removes file_name
rm -r dir_name   # removes dir_name and everything inside it
```

Daily Tool: command man for manual pages

Man(ual) pages are a form of software documentation.

They usually **document**:

- formal standards and conventions (e.g. TCP/IP)
- libraries (e.g. `stdio.h`)
- system calls (e.g. `fork()`)

They can be **accessed** via the `man` command.

Examples:

- How to use malloc?
- How to use fgets or getline?

Documentation: Useful links

```
https://www.asciitable.com/                                  # char representation in C, i.e., ASCII

https://en.wikipedia.org/wiki/Two%27s_complement             # int representation in C, i.e., 2's complement

https://www.h-schmidt.net/FloatConverter/IEEE754.html        # float/double representation in C, i.e., IEEE
                                                             standard

https://visualgo.net/en/sorting                              # Sorting (as well as other) visualisations

https://www.tutorialspoint.com/c_standard_library/index.htm # Another format for C Manual pages
```